

Introducción a OpenCV y lectura de imágenes

```
import cv2
from google.colab.patches import cv2_imshow

img = cv2.imread('imagen.jpg')
cv2_imshow(img)

print(img.shape)
```

Leer imágenes en escala de grises

```
gris = cv2.imread('imagen.jpg', cv2.IMREAD_GRAYSCALE)

cv2_imshow(gris)
print(gris.shape)
```

Conversión entre espacios de color

```
img = cv2.imread('imagen.jpg')

gris = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)

cv2_imshow(gris)
cv2_imshow(hsv)
```

Actividad: Comparar la imagen original, escala de grises y HSV.

Separar los canales HSV:

```
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
h, s, v = cv2.split(hsv)

cv2_imshow(h)
cv2_imshow(s)
cv2_imshow(v)
```

Trabajando con píxeles

```
pixel = img[100, 150]
print(pixel)
```

Cambiar un píxel:

```
img[100,150] = [0,0,255]
cv2_imshow(img)
```

Actividad:

- Cambiar varios píxeles para dibujar un pequeño punto rojo.

Aplicar slicing a listas y vectores

```
print(img[0:5,0:5])
```

Obtener una zona:

```
zona = img[100:200, 100:300]  
cv2_imshow(zona)
```

Actividad: Extraer únicamente el rostro o un objeto de la imagen.

Trabajando con píxeles (Sustitución de zonas)

```
img2 = img.copy()  
  
img2[50:150,50:150] = img[200:300,200:300]  
  
cv2_imshow(img2)
```

Traslación

```
import numpy as np  
  
filas, columnas = img.shape[:2]  
  
M = np.float32([[1,0,100],[0,1,50]])  
  
trasladada = cv2.warpAffine(img, M, (columnas, filas))  
cv2_imshow(trasladada)
```

Rotación

```
centro = (columnas//2, filas//2)  
  
M = cv2.getRotationMatrix2D(centro, 45, 1)  
rotada = cv2.warpAffine(img, M, (columnas, filas))  
  
cv2_imshow(rotada)
```

Actividad: Rotar 90°, 180° y 270°.

Escalado

```
pequena = cv2.resize(img, None, fx=0.5, fy=0.5)  
grande = cv2.resize(img, None, fx=2, fy=2)  
  
cv2_imshow(pequena)  
cv2_imshow(grande)
```

Transformación de perspectiva

```
pts1 = np.float32([[50,50],[200,50],[50,200],[200,200]])
pts2 = np.float32([[10,100],[200,50],[100,250],[250,200]])

M = cv2.getPerspectiveTransform(pts1, pts2)
perspectiva = cv2.warpPerspective(img, M, (columnas, filas))

cv2_imshow(perspectiva)
```

Actividad: Simular una imagen tomada desde otro ángulo.

Operaciones aritméticas con imágenes

```
brillo = cv2.add(img, 50)
oscura = cv2.subtract(img, 50)

cv2_imshow(brillo)
cv2_imshow(oscura)
```

Actividad: Comparar con una imagen original.

Base de la detección de movimiento

```
video = cv2.VideoCapture('video.mp4')

ret, frame1 = video.read()
ret, frame2 = video.read()

diferencia = cv2.absdiff(frame1, frame2)
cv2_imshow(diferencia)
```

Actividad: Explicar si dos frames son iguales, la diferencia, si hay movimiento

Creando nuestro propio sistema de detección de movimiento

```
while video.isOpened():
    ret, frame1 = video.read()
    ret, frame2 = video.read()

    if not ret:
        break

    diff = cv2.absdiff(frame1, frame2)
    gris = cv2.cvtColor(diff, cv2.COLOR_BGR2GRAY)
    _, th = cv2.threshold(gris, 25, 255, cv2.THRESH_BINARY)

    cv2_imshow(th)
    break
```

Actividad: Probar con un video de tránsito o personas caminando.

Introducción a la binarización de imágenes

```
gris = cv2.imread('imagen.jpg', cv2.IMREAD_GRAYSCALE)
```

```
_, binaria = cv2.threshold(gris, 127, 255, cv2.THRESH_BINARY)
cv2_imshow(binaria)
```

Umbralización binaria invertida

```
_, invertida = cv2.threshold(gris, 127, 255, cv2.THRESH_BINARY_INV)
cv2_imshow(invertida)
```

Umbralización: truncar, a cero, a cero invertido

```
_, trunc = cv2.threshold(gris, 127, 255, cv2.THRESH_TRUNC)
_, cero = cv2.threshold(gris, 127, 255, cv2.THRESH_TOZERO)
_, cero_inv = cv2.threshold(gris, 127, 255, cv2.THRESH_TOZERO_INV)

cv2_imshow(trunc)
cv2_imshow(cero)
cv2_imshow(cero_inv)
```

Umbralización de OTSU

```
_, otsu = cv2.threshold(gris, 0, 255, cv2.THRESH_BINARY+cv2.THRESH_OTSU)
cv2_imshow(otsu)
```

Explicar: El umbral se calcula automáticamente.

Umbralización adaptativa

```
adapt = cv2.adaptiveThreshold(
    gris,
    255,
    cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
    cv2.THRESH_BINARY,
    11,
    2
)

cv2_imshow(adapt)
```